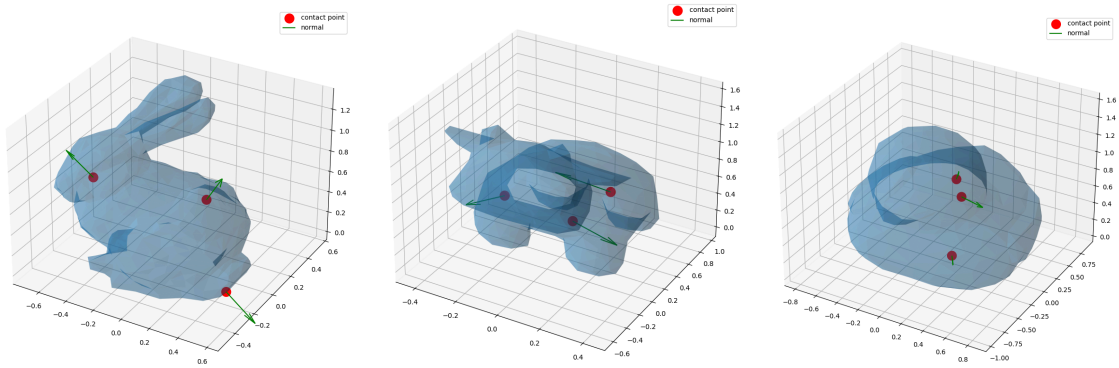
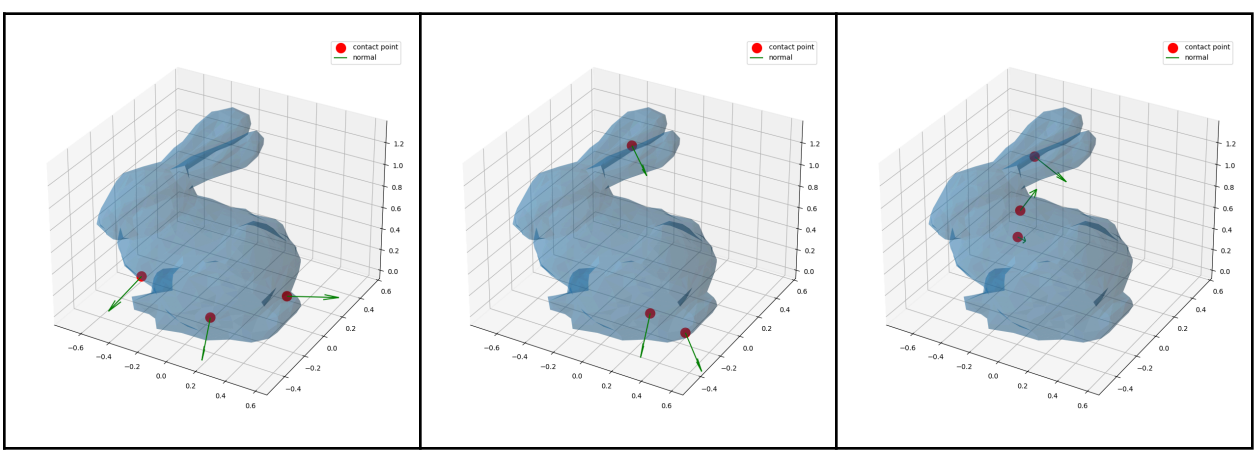
Task 2:



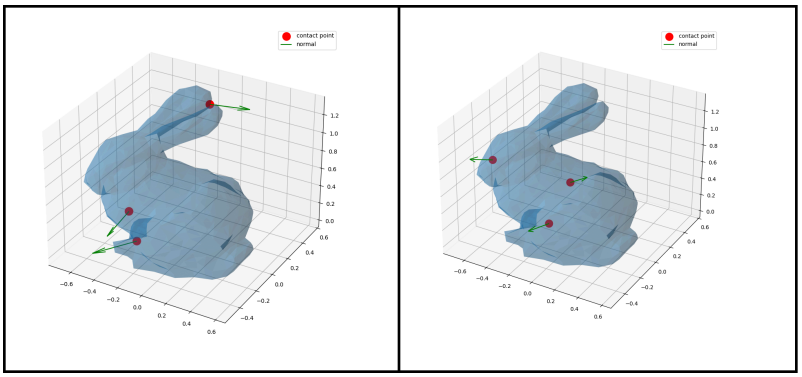
	Bunny	Cow	Duck
0.2	0.005894122289962769	-0.3870928172080204	-0.008966744595447336
0.5	0.009310443582576494	-0.5407401468279638	-0.013123909565842331
1	0.013145866090183488	-0.6699315236609699	-0.01667717774641246

As lambda increases the quality of the grasp goes away from 0. Higher lambda applies a greater torque on the contact point, which intuitively explains that if a grasp is stable, then greater torque would make it more stable, and if a grasp is unstable, then greater torque would make it less stable.

Task 3:



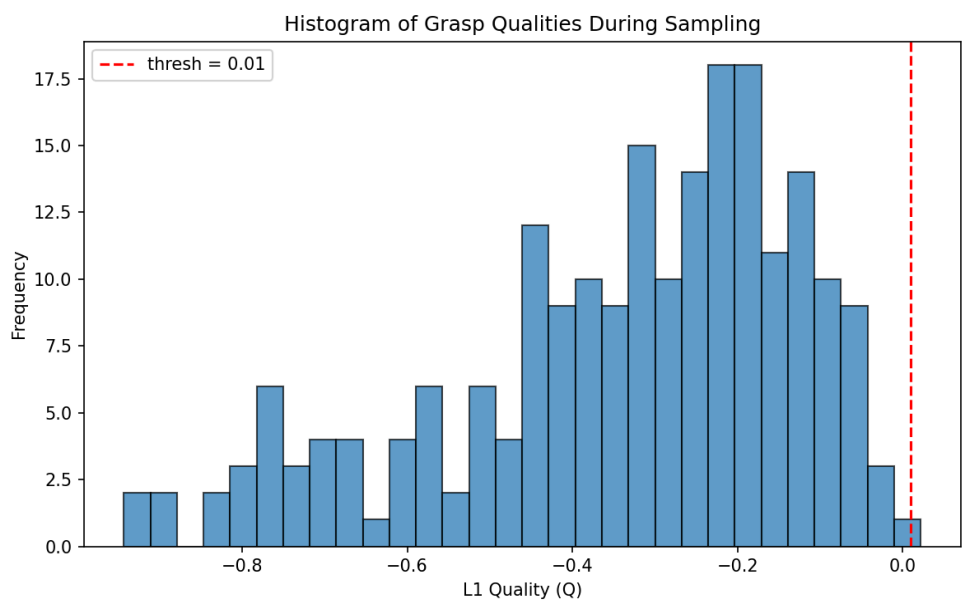
1 2 3



4 5

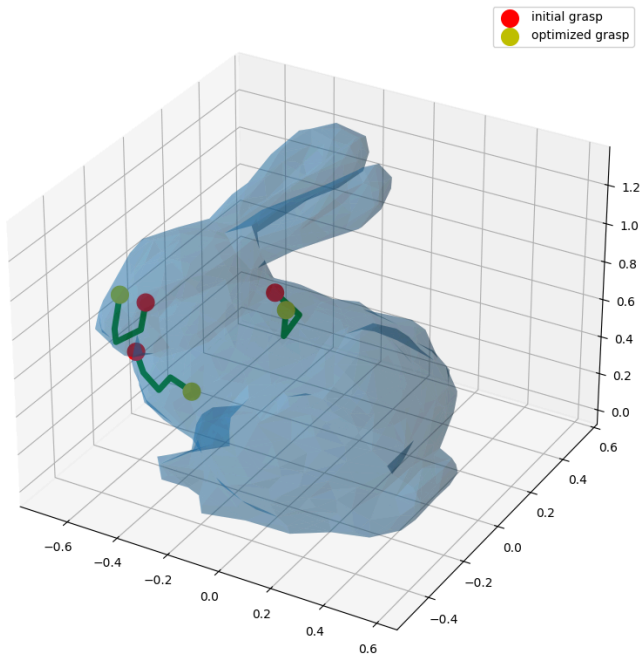
Grasp #	Grasp	Quality
1	[977, 813, 19]	-0.109662
2	[991, 768, 383]	-0.174183
3	[388, 838, 351]	-0.179855
4	[852, 893, 532]	-0.173889
5	[363, 1117, 485]	0.025369

The first 4 grasps were randomly chosen from the list of randomly sampled grasps. Grasp 5 is the final grasp that has the quality pass the threshold of 0.01.



The histogram shows that the distribution of the quality of the grasp is skewed left with the peak at around -0.2, which is consistent with the random sample of the grasps from the table.

Task 4:



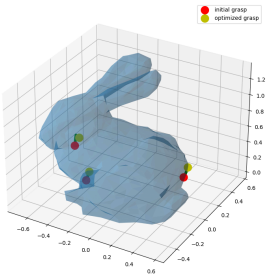
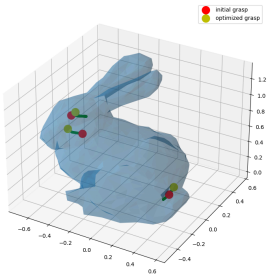
Number of quality evaluations during optimization: 14704
The quality of the given initial grasp: -0.128704
The quality of the optimized grasp: 0.018915
Trajectory Length: 5

To find the probability of finding the optimized grasp from randomly sampling the grasps, I run the following code, which was removed before the final submission:

```
# Estimate P(Q >= Q_opt) by random grasp sampling
n_faces = len(mesh.faces)
mc_samples = max(1, args.mc_samples)
random_q = np.empty(mc_samples)
for i in range(mc_samples):
    rand_grasp = list(np.random.choice(n_faces, size=3, replace=False))
    random_q[i] = alg.eval_Q(mesh, rand_grasp)
prob_ge_qopt = float(np.mean(random_q >= q_opt))
print("Estimated P(Q >= Q_opt) from %d random grasps: %.6f" % (mc_samples, prob_ge_qopt))
```

Output:
Estimated P(Q >= Q_opt) from 5000 random grasps: 0.000200

Task 5:

r=0.5	r=1
	
Initial grasp: [980, 117, 333] Initial grasp quality: 0 Optimized grasp: [565, 62, 378] Optimized quality: 0.03323	Initial grasp: [609, 253, 327] Initial grasp quality: 0.0006 Optimized grasp: [645, 394, 339] Optimized quality: 0.021351

Algorithm 1 Grasp Optimization with Reachability Constraint

Require: Mesh $\mathcal{M}$ with F faces, reachability measure r
Ensure: Optimization trajectory traj

— **Phase 1: Sample a Reachable Stable Grasp** —

repeat
  Sample 3 distinct face indices uniformly: $g \leftarrow \{f_1, f_2, f_3\} \subset \{1, \ldots, F\}$
  Compute contact points $p_i \leftarrow \text{centroid}(\mathcal{M}.\text{face}[f_i])$ for $i = 1, 2, 3$
  Compute centroid $\psi \leftarrow \frac{1}{3}\sum_{i=1}^{3} p_i$
  Compute reachability $d \leftarrow \frac{1}{3}\sum_{i=1}^{3} \|p_i - \psi\|$
  Compute quality $Q \leftarrow \text{EvalQ}(\mathcal{M}, g)$
until $d < r$ **and** $Q > 0$

— **Phase 2: Constrained Local Optimization** —

$g_{\text{curr}} \leftarrow g, \quad Q_{\text{curr}} \leftarrow Q$
traj $\leftarrow [g_{\text{curr}}]$
repeat
  for each contact f_i in g_{curr} **do**
    $\mathcal{N}_i \leftarrow \text{FindNeighbors}(\mathcal{M}, f_i, \eta = 1) \cup \{f_i\}$
  end for
  $G_{\text{opt}} \leftarrow \texttt{None}, \quad Q_{\max} \leftarrow -\infty$
  for each $(c_1, c_2, c_3) \in \mathcal{N}_1 \times \mathcal{N}_2 \times \mathcal{N}_3$ **do**
    $g' \leftarrow \{c_1, c_2, c_3\}$
    Compute contact points $p'_i \leftarrow \text{centroid}(\mathcal{M}.\text{face}[c_i])$
    Compute centroid $\psi' \leftarrow \frac{1}{3}\sum_{i=1}^{3} p'_i$
    Compute reachability $d' \leftarrow \frac{1}{3}\sum_{i=1}^{3} \|p'_i - \psi'\|$
    if $d' \geq r$ **then**
      continue ▷ Skip non-reachable candidates
    end if
    $Q' \leftarrow \text{EvalQ}(\mathcal{M}, g')$
    if $Q' > Q_{\max}$ **then**
      $Q_{\max} \leftarrow Q', \quad G_{\text{opt}} \leftarrow g'$
    end if
  end for
  if $G_{\text{opt}} = \texttt{None}$ **or** $Q_{\max} \leq Q_{\text{curr}}$ **then**
    break ▷ No improving reachable neighbor found
  end if
  $g_{\text{curr}} \leftarrow G_{\text{opt}}, \quad Q_{\text{curr}} \leftarrow Q_{\max}$
  Append g_{curr} to traj
until convergence
return traj

One improvement that I can make to this algorithm is to keep track of the visited grasps, which would improve convergence speed by removing redundant steps.